Complete Technical Manual: SOTA Crop Disease AI

1. How to Start and Run the Application

This project is a fully functional web application powered by a deep learning model. To start the application, open your terminal, navigate to the project directory, and run:

```
cd "E:\dl anigarvity\crop-disease-detection"
streamlit run app.py
```

This will start a local web server. Open your browser and navigate to http://localhost:8501. You can now upload a crop leaf image, and the application will instantly predict the disease.

2. Web Interface and Model Integration Architecture

The system bridges the gap between deep learning and user interaction through a seamless architecture:
1. The User Interface (app.py) is built with Streamlit. It handles file uploads securely in memory.
2. Once an image is uploaded, it is passed to the Inference Engine (predict.py).
3. The Inference Engine loads the pre-trained EfficientNetV2-S model ('best_model.pth') into RAM/VRAM.
4. The image undergoes standard ImageNet transformations (Resize, CenterCrop, Normalize) matching the training pipeline.
5. The model outputs raw logits, which are passed through a Softmax function to generate percentages (Confidence).
6. The top predictions are returned as a JSON/Dictionary back to the UI, which dynamically renders a bar chart using Pandas.

3. Dataset Preparation (dataset.py)

Purpose: This script completely automates the downloading and processing of the massive 87,000+ image dataset.
How it works:
- Uses 'kagglehub' to securely bypass authentication limits and download the 'vipoooool/new-plant-diseases-dataset'.
- Recursively walks through the downloaded folders to find the exact plant categories (38 classes).
- Gathers all JPG/PNG files, removes duplicates, and shuffles them.
- Splits them strictly into 80% Training, 10% Validation, and 10% Testing sets.
- Copies them to 'data/processed' ensuring a pristine, isolated dataset.

```
import os
import shutil
import random
from pathlib import Path
from tqdm import tqdm
import kagglehub

def prepare_dataset(dest_dir, split_ratios=(0.8, 0.1, 0.1), seed=42):
    random.seed(seed)

    print("Downloading the comprehensive 38-class dataset via kagglehub...")
    # This dataset contains 38 classes and ~87K images
    path = kagglehub.dataset_download('vipoooool/new-plant-diseases-dataset')

    print(f"Dataset downloaded to: {path}")

    # Locate the directories containing the actual class folders
        # Usually it's in a folder like 'New Plant Diseases Dataset(Augmented)/New Plant Diseases
Dataset(Augmented)/train'
    base_raw_dirs = []
    for root, dirs, files in os.walk(path):
```

```
        # Identify directory containing plant classes
        if any(d.lower().startswith('apple___') or d.lower().startswith('tomato___') for d in dirs):
            base_raw_dirs.append(root)

    if not base_raw_dirs:
           raise FileNotFoundError("Could not find the dataset images folder inside the downloaded kagglehub
dataset.")

    # We will gather images from all found directories (train/valid splits inside the zip)
    # and re-split them ourselves to ensure consistency.

    dest_path = Path(dest_dir)
    train_dir = dest_path / 'train'
    val_dir = dest_path / 'val'
    test_dir = dest_path / 'test'

    for d in [train_dir, val_dir, test_dir]:
        d.mkdir(parents=True, exist_ok=True)

    # Gather all images across all matching base directories
    all_classes = set()
    for base_dir in base_raw_dirs:
        for d in Path(base_dir).iterdir():
            if d.is_dir():
                all_classes.add(d.name)

    print(f"Found {len(all_classes)} distinct classes.")

    # Dictionary to hold all image paths per class
    class_images = {cls: [] for cls in all_classes}

    print("Gathering image paths...")
    for base_dir in base_raw_dirs:
        for cls in all_classes:
            cls_path = Path(base_dir) / cls
            if cls_path.exists():
                                    images  =  list(cls_path.glob('*.jpg'))  +  list(cls_path.glob('*.JPG'))  +
list(cls_path.glob('*.png'))
                class_images[cls].extend(images)

    for cls in tqdm(all_classes, desc="Processing and Splitting classes"):
        (train_dir / cls).mkdir(exist_ok=True)
        (val_dir / cls).mkdir(exist_ok=True)
        (test_dir / cls).mkdir(exist_ok=True)

        images = class_images[cls]
        # Remove duplicates if any (by filename/path)
        images = list(set(images))
        images = sorted(images)
        random.shuffle(images)

        n_total = len(images)
        n_train = int(n_total * split_ratios[0])
        n_val = int(n_total * split_ratios[1])

        train_imgs = images[:n_train]
        val_imgs = images[n_train:n_train+n_val]
        test_imgs = images[n_train+n_val:]

        for img in train_imgs:
            shutil.copy2(img, train_dir / cls / img.name)
        for img in val_imgs:
            shutil.copy2(img, val_dir / cls / img.name)
```

```
        for img in test_imgs:
            shutil.copy2(img, test_dir / cls / img.name)

    print("Dataset preparation complete! Extracted to:", dest_dir)

if __name__ == '__main__':
    dest = 'data/processed'
    if not os.path.exists(os.path.join(dest, 'train')):
        prepare_dataset(dest)
    else:
        print("Dataset already processed.")
```

Complete Technical Manual: SOTA Crop Disease AI

4. Deep Learning Model (model.py)

Purpose: Defines the State-of-the-Art network architecture.
How it works:
- Imports EfficientNetV2-S directly from torchvision with pre-trained ImageNet weights.
- We dynamically locate the final classification head (model.classifier[1]) and replace it with a new nn.Linear layer outputting exactly 38 classes.
- EfficientNetV2 is fundamentally superior here because it uses Fused-MBConv layers (replacing standard Depthwise convolutions in early stages) which massively accelerates training while reducing parameter count.

```
import torch
import torch.nn as nn
from torchvision.models import efficientnet_v2_s, EfficientNet_V2_S_Weights

def get_model(num_classes):
    # Using State-of-the-Art EfficientNetV2-S
    # It has excellent parameter efficiency, faster training, and higher accuracy than older models like ResNet
or standard EfficientNet.
    weights = EfficientNet_V2_S_Weights.DEFAULT
    model = efficientnet_v2_s(weights=weights)

    # Replace the final classification head
    in_features = model.classifier[1].in_features
    model.classifier[1] = nn.Linear(in_features, num_classes)

    return model

if __name__ == '__main__':
    model = get_model(38)
    print("SOTA Model initialized:", model.__class__.__name__)
    total_params = sum(p.numel() for p in model.parameters())
    print(f"Total Parameters: {total_params:,}")
```

Complete Technical Manual: SOTA Crop Disease AI

5. Training Pipeline & Advanced Engineering (train.py)

Purpose: Trains the network using highly optimized SOTA techniques.
How it works:
- Automatic Mixed Precision (AMP): Uses 'torch.amp.autocast' and 'GradScaler'. This forces calculations into 16-bit floats instead of 32-bit where safe. Result: 2x faster training, 50% less GPU memory.
- OneCycleLR Scheduler: Instead of a flat learning rate, this ramps the LR up quickly and then cosine-decays it down. This enables 'Super-Convergence'.
- Label Smoothing (0.1): Changes the absolute '1.0' target to '0.9'. Prevents the model from being overconfident, acting as severe regularization.
- Optimizer: Uses AdamW instead of Adam, offering mathematically correct weight decay decoupling.
- Augmentations: RandomResizedCrop, Horizontal/Vertical Flips, and ColorJittering force the model to learn shapes, not specific pixels.

```
import os
import json
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
from tqdm import tqdm

from src.model import get_model

def train():
    device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
    print(f"Training on device: {device}")

    data_dir = 'data/processed'
    if not os.path.exists(data_dir):
        raise FileNotFoundError(f"Processed data directory not found: {data_dir}. Run src/dataset.py first.")

    # Advanced Data Augmentation for SOTA training
    train_transforms = transforms.Compose([
        transforms.RandomResizedCrop(224),
        transforms.RandomHorizontalFlip(),
        transforms.RandomVerticalFlip(),
        transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2, hue=0.1),
        transforms.ToTensor(),
        transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
    ])

    val_transforms = transforms.Compose([
        transforms.Resize(256),
        transforms.CenterCrop(224),
        transforms.ToTensor(),
        transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
    ])

    # Load Datasets
    train_dataset = datasets.ImageFolder(os.path.join(data_dir, 'train'), train_transforms)
    val_dataset = datasets.ImageFolder(os.path.join(data_dir, 'val'), val_transforms)

    class_names = train_dataset.classes
    with open('class_names.json', 'w') as f:
        json.dump(class_names, f)

    # Hyperparameters
```

```
    batch_size = 64  # Good balance for AMP on 24GB/16GB/8GB GPUs
    num_epochs = 10  # Reduced slightly to keep prototype time reasonable, but enough with OneCycleLR
    learning_rate = 1e-3

          train_loader   =   DataLoader(train_dataset,   batch_size=batch_size,   shuffle=True,   num_workers=4,
pin_memory=True)
    val_loader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False, num_workers=4, pin_memory=True)

    # Model Setup
    model = get_model(len(class_names)).to(device)

    # Label Smoothing improves generalization for large datasets
    criterion = nn.CrossEntropyLoss(label_smoothing=0.1)

    # AdamW is generally better than Adam for models with weight decay
    optimizer = optim.AdamW(model.parameters(), lr=learning_rate, weight_decay=1e-4)

    # OneCycleLR is SOTA for fast convergence
    scheduler = optim.lr_scheduler.OneCycleLR(
        optimizer, max_lr=learning_rate,
        steps_per_epoch=len(train_loader),
        epochs=num_epochs
    )

    # AMP Scaler for mixed precision
    scaler = torch.amp.GradScaler('cuda' if torch.cuda.is_available() else 'cpu')

    best_val_acc = 0.0
    train_losses, val_losses, train_accs, val_accs = [], [], [], []

    print(f"Starting training for {num_epochs} epochs with {len(class_names)} classes...")

    for epoch in range(num_epochs):
        model.train()
        running_loss = 0.0
        correct = 0
        total = 0

        # Training loop with AMP
        train_bar = tqdm(train_loader, desc=f"Epoch {epoch+1}/{num_epochs} [Train]")
        for inputs, labels in train_bar:
            inputs, labels = inputs.to(device), labels.to(device)

            optimizer.zero_grad()

            with torch.amp.autocast('cuda' if torch.cuda.is_available() else 'cpu'):
                outputs = model(inputs)
                loss = criterion(outputs, labels)

            scaler.scale(loss).backward()
            scaler.step(optimizer)
            scaler.update()
            scheduler.step()

            running_loss += loss.item()
            _, predicted = outputs.max(1)
            total += labels.size(0)
            correct += predicted.eq(labels).sum().item()

            train_bar.set_postfix({'loss': f'{loss.item():.4f}'})

        train_loss = running_loss / len(train_loader)
        train_acc = 100. * correct / total
```

```
        # Validation loop
        model.eval()
        val_loss = 0.0
        correct = 0
        total = 0

        with torch.no_grad():
            for inputs, labels in tqdm(val_loader, desc=f"Epoch {epoch+1}/{num_epochs} [Val]"):
                inputs, labels = inputs.to(device), labels.to(device)

                with torch.amp.autocast('cuda' if torch.cuda.is_available() else 'cpu'):
                    outputs = model(inputs)
                    loss = criterion(outputs, labels)

                val_loss += loss.item()
                _, predicted = outputs.max(1)
                total += labels.size(0)
                correct += predicted.eq(labels).sum().item()

        val_loss = val_loss / len(val_loader)
        val_acc = 100. * correct / total

        print(f"Epoch {epoch+1} - Train Loss: {train_loss:.4f} Acc: {train_acc:.2f}% | Val Loss: {val_loss:.4f}
Acc: {val_acc:.2f}%")

        train_losses.append(train_loss)
        val_losses.append(val_loss)
        train_accs.append(train_acc)
        val_accs.append(val_acc)

        if val_acc > best_val_acc:
            best_val_acc = val_acc
            torch.save(model.state_dict(), 'models/best_model.pth')
            print("Best model saved!")

    # Save metrics
    metrics = {
        'train_losses': train_losses,
        'val_losses': val_losses,
        'train_accs': train_accs,
        'val_accs': val_accs
    }
    with open('results/metrics.json', 'w') as f:
        json.dump(metrics, f)

if __name__ == '__main__':
    train()
```

Complete Technical Manual: SOTA Crop Disease AI

6. Inference Engine (predict.py)

Purpose: Bridges the trained model weights and raw image input for production.
How it works:
- Loads 'class_names.json' generated during training so predictions map to human-readable names (e.g., 'Tomato_Early_Blight').
- Loads the model weights strictly in evaluation mode (model.eval()), turning off Dropout/BatchNorm updates.
- Uses torch.no_grad() to completely disable the autograd engine, saving immense memory and drastically speeding up predictions.
- Uses torch.topk to extract the 3 highest probability classes.

```
import json
import torch
import torch.nn.functional as F
from torchvision import transforms
from PIL import Image
import numpy as np

from src.model import get_model
from src.treatments import get_treatment_details
from src.gradcam import GradCAM

class CropDiseasePredictor:
    def __init__(self, model_path='models/best_model.pth', class_names_path='class_names.json'):
        self.device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

        with open(class_names_path, 'r') as f:
            self.class_names = json.load(f)

        self.model = get_model(len(self.class_names)).to(self.device)
        self.model.load_state_dict(torch.load(model_path, map_location=self.device))
        self.model.eval()

        # Initialize Grad-CAM
        self.gradcam = GradCAM(self.model)

        self.transform = transforms.Compose([
            transforms.Resize(256),
            transforms.CenterCrop(224),
            transforms.ToTensor(),
            transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
        ])

    def validate_leaf_presence(self, pil_image: Image.Image):
        """
        Heuristic Out-Of-Distribution check to verify if the uploaded image
        has natural botanical color and saturation characteristics.
        """
        img_rgb = pil_image.convert('RGB')
        img_hsv = img_rgb.convert('HSV')
        _, s, v = img_hsv.split()

        s_arr = np.array(s) / 255.0
        v_arr = np.array(v) / 255.0

        avg_sat = float(np.mean(s_arr))
        white_bg_ratio = float(np.mean((v_arr > 0.70) & (s_arr < 0.15)))

        # Document, screenshot, or grayscale filter detection
        if avg_sat < 0.12 and white_bg_ratio > 0.55:
```

```
            return False, "Non-plant image detected (appears to be a document, paper, or text screenshot)."
        if avg_sat < 0.08:
             return False, "Low color saturation detected. Living plant leaves require green, yellow, or brown
pigment."

        return True, "Valid plant leaf characteristics verified."

    def predict(self, image_input):
        """
        Runs comprehensive diagnosis:
        - Image validation
        - Neural classification
        - Grad-CAM visual attention mapping
        - Treatment extraction
        """
        if isinstance(image_input, Image.Image):
            pil_image = image_input.convert('RGB')
        else:
            pil_image = Image.open(image_input).convert('RGB')

        is_leaf, validity_msg = self.validate_leaf_presence(pil_image)

        # Preprocess tensor for model
        tensor = self.transform(pil_image).unsqueeze(0).to(self.device)
        tensor.requires_grad = True

        # Model forward pass
        outputs = self.model(tensor)
        probs = F.softmax(outputs, dim=1)[0]

        top_probs, top_indices = torch.topk(probs, 3)

        top_predictions = []
        for i in range(3):
            cls_key = self.class_names[top_indices[i].item()]
            clean_display = cls_key.replace('___', ' - ').replace('_', ' ')
            top_predictions.append({
                'class': cls_key,
                'clean_name': clean_display,
                'confidence': float(top_probs[i].item() * 100)
            })

        best_class = top_predictions[0]['class']
        best_idx = top_indices[0].item()

        # Generate Grad-CAM attention heatmap
        try:
            cam_map, _ = self.gradcam.generate_heatmap(tensor, target_class_idx=best_idx)
            gradcam_image = self.gradcam.overlay_heatmap(pil_image, cam_map, alpha=0.45)
        except Exception as e:
            print(f"Grad-CAM generation error: {e}")
            gradcam_image = pil_image

        # Get structured treatment data
        treatment_info = get_treatment_details(best_class)

        return {
            'is_valid_leaf': is_leaf,
            'validity_message': validity_msg,
            'top_predictions': top_predictions,
            'treatment': treatment_info,
            'gradcam_image': gradcam_image,
            'original_image': pil_image
```

```
        }
```

Complete Technical Manual: SOTA Crop Disease AI

7. User Interface (app.py)

Purpose: The front-facing web application.
How it works:
- Built purely in Python using Streamlit.
- Uses '@st.cache_resource' to load the predictor globally. This means the heavy model is only loaded into memory once when the server starts, not every time a user uploads an image. This makes predictions instantaneous.
- Provides a polished UI utilizing columns, success banners, and a dynamic Pandas dataframe converted into a Streamlit Bar Chart.

```
import os
import streamlit as st
from PIL import Image
import pandas as pd

from src.predict import CropDiseasePredictor
from src.report_generator import generate_pdf_report

st.set_page_config(
    page_title="CropSense AI - Smart Plant Pathology",
    page_icon="",
    layout="wide"
)

# Custom CSS for polished agritech dashboard styling
st.markdown("""
<style>
    .metric-card {
        background-color: #f0fdf4;
        border: 1px solid #86efac;
        padding: 1.2rem;
        border-radius: 0.75rem;
        margin-bottom: 1rem;
    }
    .warning-card {
        background-color: #fffbeb;
        border: 1px solid #fde68a;
        padding: 1rem;
        border-radius: 0.5rem;
        margin-bottom: 1rem;
    }
    .stButton>button {
        background-color: #16a34a;
        color: white;
        font-weight: 600;
        border-radius: 0.5rem;
        padding: 0.6rem 1.5rem;
    }
    .stButton>button:hover {
        background-color: #15803d;
        color: white;
    }
</style>
""", unsafe_allow_html=True)

@st.cache_resource
def load_predictor():
    if not os.path.exists('models/best_model.pth') or not os.path.exists('class_names.json'):
        return None
    return CropDiseasePredictor()
```

```
predictor = load_predictor()

# App Header
st.title(" CropSense AI  Plant Disease Diagnostic Hub")
st.caption("Powered by EfficientNetV2-S (38 Botanical Classes) | Grad-CAM Explainable AI | Real-Time Agronomic
Remedies")

if predictor is None:
    st.error("Model checkpoints or class mappings not found. Please verify models/best_model.pth exists.")
    st.stop()

# Input Selection Tabs
tab_upload, tab_camera = st.tabs([" Upload Leaf Image", " Live Camera Capture"])
input_image = None

with tab_upload:
    uploaded_file = st.file_uploader(
        "Select a leaf photo from your device (JPG, JPEG, PNG)",
        type=["jpg", "jpeg", "png"],
        key="uploader"
    )
    if uploaded_file is not None:
        input_image = Image.open(uploaded_file)

with tab_camera:
    camera_file = st.camera_input("Snap a live photo of a plant leaf in the field", key="camera")
    if camera_file is not None:
        input_image = Image.open(camera_file)

if input_image is not None:
    st.markdown("---")

    # Run Prediction Button
    col_btn, _ = st.columns([1, 4])
    with col_btn:
        run_analysis = st.button(" Diagnose Plant Health", use_container_width=True)

    if run_analysis:
        with st.spinner("Analyzing botanical morphology and lesion patterns with EfficientNetV2..."):
            diagnosis = predictor.predict(input_image)

        top_pred = diagnosis['top_predictions'][0]
        treatment = diagnosis['treatment']

        # 1. Leaf Gatekeeper / Out-of-Distribution Warning
        if not diagnosis['is_valid_leaf']:
            st.error(f" **Non-Plant Image Warning:** {diagnosis['validity_message']}")
                st.info("The AI model was trained strictly on plant leaves. Uploading documents, screens, or
non-botanical images will result in speculative classification.")
        elif top_pred['confidence'] < 75.0:
                 st.warning(f" **Moderate Confidence ({top_pred['confidence']:.1f}%):** The lesion pattern is
ambiguous or obscured. Consider taking a closer photo under even natural lighting.")

        # 2. Main Dashboard Layout
        col_visuals, col_details = st.columns([1, 1], gap="large")

        with col_visuals:
            st.subheader(" Visual Inspection & Explainable AI")

            img_tabs = st.tabs(["Original Specimen", " Grad-CAM Attention Heatmap"])
            with img_tabs[0]:
                                    st.image(diagnosis['original_image'],  caption="Uploaded  Leaf  Specimen",
```

```
use_container_width=True)
            with img_tabs[1]:
                 st.image(diagnosis['gradcam_image'], caption="Grad-CAM Activation: Red & yellow zones pinpoint
the exact leaf lesions and patterns the model analyzed.", use_container_width=True)

        with col_details:
            st.subheader(" Diagnostic Verdict")

            # Highlight Banner
            if "healthy" in top_pred['class'].lower():
                st.success(f"###  {treatment['crop']}  Healthy")
                st.write(f"**Confidence:** {top_pred['confidence']:.1f}%")
            else:
                st.error(f"###  {treatment['crop']}  {treatment['disease']}")
                                st.write(f"**Confidence:**  {top_pred['confidence']:.1f}%    |    **Pathogen:**
{treatment['pathogen']}")

            st.markdown("#### Top Probabilities")
            df = pd.DataFrame(diagnosis['top_predictions'])
            df_chart = df.set_index('clean_name')[['confidence']]
            st.bar_chart(df_chart, height=220)

        # 3. Actionable Treatment & Agronomic Solutions
        st.markdown("---")
        st.subheader(" Actionable Treatment & Crop Management Plan")

        t_col1, t_col2 = st.columns(2, gap="medium")

        with t_col1:
            with st.expander(" Observed Symptoms & Pathogen Details", expanded=True):
                st.write(f"**Pathogen:** {treatment['pathogen']}")
                st.write(f"**Typical Symptoms:** {treatment['symptoms']}")

            with st.expander(" Organic & Biological Remedies", expanded=True):
                st.write(treatment['organic'])

        with t_col2:
            with st.expander(" Recommended Chemical Controls", expanded=True):
                st.write(treatment['chemical'])

            with st.expander(" Long-Term Cultural Prevention", expanded=True):
                st.write(treatment['prevention'])

        # 4. Exportable PDF Health Report
        st.markdown("---")
        st.subheader(" Download Official Diagnostic Report")

        with st.spinner("Compiling high-resolution PDF diagnostic report..."):
            pdf_bytes = generate_pdf_report(
                diagnosis['original_image'],
                diagnosis['gradcam_image'],
                diagnosis
            )

        file_name = f"CropSense_Report_{treatment['crop']}_{treatment['disease'].replace(' ', '_')}.pdf"
        st.download_button(
            label=" Download Diagnostic PDF Report",
            data=pdf_bytes,
            file_name=file_name,
            mime="application/pdf",
            use_container_width=False
        )
```

```
else:
    # Empty State Guidance
    st.info(" Upload a clear leaf photo or take a picture using your camera above to begin instant diagnosis.")

    with st.expander(" Supported Crops & Diseases (Click to view full coverage)"):
        st.markdown("""
          **Covered Crops (14 species):** Apple, Blueberry, Cherry, Corn (Maize), Grape, Orange, Peach, Bell Pepper, Potato, Raspberry, Soybean, Squash, Strawberry, Tomato.
          **Detection Capabilities:** 38 health and disease categories including Early/Late Blights, Powdery Mildews, Rusts, Bacterial Spots, Black Rots, Citrus Greening, Spider Mites, and Viral infections.
        """)
```